

MEMORAQ: The Strategy Co-Pilot Where the LLM Is Never Allowed to Do the Math

The most valuable design rule in MEMORAQ is not a clever prompt. It is a hard boundary: the language model is never allowed to compute a number that appears in the product.

What the system does

MEMORAQ is a strategic intelligence platform for a B2B analytics company - in our corpus, a firm called Northwind Instruments. It keeps a persistent memory of the organization's own decisions and their outcomes, tracks competitor moves, runs deterministic what-if simulations, and answers executive questions with labeled evidence. Then it stops and hands the decision to a human. The loop we built the whole product around is: REMEMBER -> UNDERSTAND -> SIMULATE -> EXPLAIN -> DECIDE -> LEARN.

The stack is deliberately boring. A React 19 + TypeScript front end on Vite, styled like a Swiss editorial page - grid-paper background, square corners, red and ink accents, tabular numerals, because executives actually read the numbers. The backend is Convex: queries and mutations for dashboard state, node actions for anything that leaves the process. Memory is [open-source Hindsight](#), self-hosted next to the app. No API key, no vendor round-trip - the client just points at `HINDSIGHT_BASE_URL`.

We run two Hindsight banks. `northwind-experience` holds the company's own record: every strategic decision with its situation, options, expected outcome, actual outcome, and the lesson extracted after the fact. `northwind-world` holds competitive intelligence: competitor pricing changes, launches, partnerships, and market context. Each bank is created with a mission string that shapes how Hindsight extracts facts - the experience bank's reflect mission literally says "cite what actually happened, and never invent numbers."

That persistent layer matters more than any model choice. A context window forgets the moment the conversation ends; organizational memory should outlive every conversation. If you haven't thought hard about the difference, Vectorize's piece on [what agent memory actually is](#) is the clearest framing I've found.

Everything Hindsight-related lives in one file, `hindsight_client.ts`. Every write goes through `retain` with a `documentId`, so re-ingesting the corpus is an upsert, not a duplicate. Every read is one of the three verbs from the [Hindsight documentation](#): `recall` for ranked memories, `reflect` for an LLM synthesis grounded in those memories. Ingestion runs with `async: true` so Hindsight's fact-extraction worker never blocks a request.

The rule: the LLM never does the math

Here is the opinionated part. Most "AI analyst" systems I've seen fail not because the model is dumb, but because nobody can tell which numbers in the answer are real. An executive will screenshot a figure and put it in a board deck. If the model computed that figure, you have shipped a hallucination with nice formatting.

So MEMORAQ has exactly two compute surfaces, and neither is an LLM:

- `financials.ts` - pure functions that turn six months of raw monthly figures into KPIs: revenue,

margins, churn, CAC, month-over-month deltas, anomalies.

- `simulation.ts` - pure functions that turn five scenario levers (price, discount, marketing, volume, hiring) into projected revenue, costs, and profit.

Both are ordinary TypeScript with no I/O and no model. `agent.ts` orchestrates the rest: it classifies the question with regexes, recalls the right Hindsight banks, runs the simulator when the question contains a scenario, and only then calls `reflect` so the LLM can explain - over memory and deterministic numbers, never over arithmetic.

Classification is intentionally crude:

```
export function classifyQuestion(q: string): QuestionIntent {
  const s = q.toLowerCase();
  const asksHistory = /(last time|previously|history|historical|last year|past|
before)/.test(s);
  const asksPricing = /(price|pricing|discount|cut|reduc)/.test(s);
  const asksSim = /(what if|simulate|scenario|would)/.test(s);

  if (asksSim && /\d/.test(s)) return "simulation";
  if (asksHistory && asksPricing) return "pricing_history";
  // ... competitor, profit_analysis, lessons elided
  return "general";
}
```

A regex can't reason, but it also can't surprise you. The intent decides which banks get recalled and whether the simulator runs, and I can read the whole routing table in one screen. I'd rather extend this by hand than debug a classifier I can't explain.

The LLM's moment comes at `reflect`, and even there I hand it the deterministic numbers with instructions not to recompute them:

```
const response = await client.reflect(bankId, opts.query, {
  budget: "low",
  includeFacts: true,
});
const memories = response.based_on?.memories ?? [];
const sources: EvidenceSource[] = memories.map((m) => ({
  text: m.text,
  type: m.type ?? "memory",
  date: (m.occurred_start || "").slice(0, 10),
  kind: opts.bank,
}));
```

`includeFacts: true` is the feature that makes the whole product honest: every `reflect` answer comes back with the exact memories it was grounded in, and those become the citations in the UI. The orchestrator's `reflect` query includes context like "use these numbers as given, do not recompute," so the synthesis layer inherits numbers instead of generating them.

The simulator is the second surface, and it is four lines of arithmetic plus a stack of printed assumptions:

```
// simulation.ts – the entire demand model
const netPriceFactor = (1 + p.priceDeltaPct / 100) * (1 - p.discountPct / 100);
const elasticity = -1.2; // calibrated from decision D-101
const demandFactor =
  Math.pow(netPriceFactor, elasticity) * (1 + p.volumeDeltaPct / 100);
const newUnits = baselineUnits * demandFactor;
const newRevenue = newUnits * newPrice;

const assumptions: string[] = [
  "Baseline: August 2026 actuals (latest closed month).",
  "Demand elasticity -1.2 on net price, calibrated from D-101 (a 5% cut lifted volume 6.8%);",
  "the >6% region was never tested, so treat large cuts as an extrapolation.",
  "No competitor response modeled – historical pattern (D-202, May 2026) suggests rivals",
  "react to price moves within ~4 weeks.",
];
```

Two things to notice. First, the elasticity is calibrated from our own history, not a textbook: D-101 was a 5% price cut on Product A that lifted volume 6.8% while giving up only 1.1 margin points. Second, the assumptions array is part of the return value. The UI renders it next to every simulation. The model cannot drop it, soften it, or forget it, because the model never touches it.

Anomaly detection is dumb on purpose too - a 5% month-over-month threshold and sentence composition:

```
if (cur.momRevenuePct !== null && Math.abs(cur.momRevenuePct) >= ANOMALY_THRESHOLD)
  parts.push(`revenue ${cur.momRevenuePct > 0 ? "grew" : "declined"}
    ${Math.abs(cur.momRevenuePct)}%`);
if (cur.momMarketingPct !== null && Math.abs(cur.momMarketingPct) >= ANOMALY_THRESHOLD)
  parts.push(`marketing spend ${cur.momMarketingPct > 0 ? "increased" : "decreased"}
    ${Math.abs(cur.momMarketingPct)}%`);
if (parts.length >= 2 || (parts.length === 1 && severity === "high")) {
  const statement = `In ${cur.label}, ${parts[0]} while ${parts[1]}`;
  anomalies.push({ monthIndex: cur.monthIndex, label: cur.label, severity, statement, ... });
}
```

That code produces the single most useful sentence in the product: *"In Jun 2026, revenue declined 11.4% while marketing spend increased 8.2%."* No model required, no model could get it more right.

What it looks like in practice

Take the sample question our executives ask most: **"What happened the last time we reduced Product A's price?"** The classifier routes it to `pricing_history`, `recall` hits the experience bank, and `Hindsight` returns the retained records for D-101 (September 2025: 5% cut, units +6.8%, win rate 31% -> 38%, margin -1.1 points) and, because recall ranks by relevance rather than exact match, D-102 (November 2025: a 10% cut on Product B that recovered renewals but permanently ceded 6.2 margin points). The reflect answer weaves both into a paragraph, and every claim in it carries a date and a "Historical Fact" label.

Now the scenario question: **"What if we reduce the price by 7%?"** `parseSimParams` extracts the -7% from the sentence - again with regex - and `runSimulation` does the rest: units up about 9%, revenue up only about 1.5%, and profit down roughly 78%, because the extra units drag COGS and operations up by ~6% while net price is only 93% of baseline. That is a counterintuitive answer executives genuinely need to see, and it arrives with the label "Deterministic simulation - not a prediction" and the assumptions attached, including the admission that the >6% cut region was never tested. A confident-sounding LLM would have produced a plausible number. This one is auditable.

Competitor questions go to the world bank, and the interpretation layer is rule-based rather than generative:

hypotheses like "Apex may extend promotional pricing next quarter - confidence: medium" with the evidence listed line by line (three public pricing actions in roughly 90 days). Every output is explicitly a hypothesis. The status bar across the app uses seven evidence kinds - Historical Fact, Current Data, Simulation, AI Interpretation, Hypothesis, Assumption, World Memory - so at any moment you can tell what kind of claim you're reading.

And when Hindsight is unreachable, the system does not pretend. A health probe (`getVersion` with a 2.5-second timeout, cached for 20 seconds) detects the outage once per action, not once per call. Operations fall back to a local store searched by keyword overlap, answers are prefixed with a visible **[Fallback mode]** banner, and every operation - retain, recall, reflect - is logged to a `memoryActivity` table that drives a live Memory Inspector showing the bank, the query, the sources, and the mode. Nothing is faked as live.

Lessons learned

1. Give every memory write an identity. The `documentId` on `retain` makes ingestion idempotent - re-seeding the corpus after a schema change is a no-op instead of a duplicated memory. This sounds minor until the first time you re-run an ingest pipeline and realize you've polluted your own history.

2. Log every memory operation and show it to users. The `memoryActivity` table is the most-debugged feature in the codebase. When an answer is wrong, the first question is always "what did memory return?" - and having kind, bank, query, sources, and mode for every operation turns a mystery into a lookup. Memory you can't observe is memory you can't debug.

3. Labeled degradation beats fake uptime. It is tempting to hide a memory-layer outage behind generic answers. Don't. The fallback store, the red banner, and the `mode` field on every log entry cost a day to build and have earned permanent trust - users know the system tells them when it's guessing.

4. Separate your runtimes early. In Convex, node actions ("`use node`") and isolate queries/mutations can't share a file, `ActionCtx` is the type you actually get inside actions, and pure helpers must not import the generated API or you create circular type inference (which is why the hypothesis engine lives in its own `hypotheses.ts`). Learning each of these mid-refactor is painful; a one-hour layout pass at the start avoids all three.

5. Calibrate from your own history, and print the provenance. The simulator's elasticity of -1.2 comes from exactly one real experiment, and every simulation says so - including the fact that the deeper-cut region was never tested. Skeptical users trust a system that admits "extrapolation" far more than one that always sounds confident.

Remember the past, understand the present, simulate the future, let humans decide - the tagline wrote itself once the architecture did. What made the system trustworthy wasn't anything we added to the model. It was the subtraction: deterministic code computes, Hindsight remembers, and the LLM only explains.